Parallel Computation/Program Issues

Parallel Programs: Definitions

- **A parallel program is comprised of a number of tasks running as threads (or processes) on a number of processing elements that cooperate/communicate as part of a single parallel computation.**
- ***Task*:**
  - **Arbitrary piece of undecomposed work in parallel computation**
  - **Executed sequentially on a single processor; concurrency in parallel computation is only across tasks.**
- ***Parallel or Independent Tasks:***
  - **Tasks that with no dependencies among them and thus can run in parallel on different processing elements.**
- ***Process (thread)*:** A process is a running instance of a program.
  - **Abstract program entity that performs the computations assigned to a task**
  - **Processes communicate and synchronize to perform their tasks**
- ***Processor or (Processing Element)*:**
  - **Physical computing engine on which a process executes sequentially**
  - **Processes virtualize machine to programmer**

Factors Affecting Parallel System Performance

- **Parallel Algorithm Related:**
  - Available concurrency and profile, uniformity, patterns.
  - Type of parallelism present: Functional and/or data parallelism.
  - Required communication/synchronization, uniformity and patterns.
  - Data size requirements.
  - Communication to computation ratio (C-to-C ratio, lower is better).
- **Parallel program Related:**
  - Programming model used.
  - Resulting data/code memory requirements, locality and working set characteristics.
  - Parallel task grain size.
  - Assignment (mapping) of tasks to processors: Dynamic or static.
  - Cost of communication/synchronization primitives.
- **Hardware/Architecture related:**
  - Total CPU computational power available.
  - Types of computation modes supported.
  - Shared address space Vs. message passing.
  - Communication network characteristics (topology, bandwidth, latency)
  - Memory hierarchy properties.

Dependency Analysis & Conditions of Parallelism

- **Dependency analysis is concerned with detecting the presence and type of dependency between tasks that prevent tasks from being independent and from running in parallel on different processors and can be applied to tasks of any grain size.**

- **Dependencies between tasks can be 1- algorithm/program related or 2- hardware resource/architecture related.**

- **Algorithm/program Task Dependencies:**
  - **Data Dependence:**
    - **True Data or Flow Dependence** Write first Read later = True dependence
    - **Name Dependence:**
      - **Anti-dependence** Read first Write later = Anti-dependence
      - **Output (or write) dependence** This is also called Write After Write (WAW).
  - **Control Dependence** A statement may depend on a condition
- **Hardware/Architecture Resource Dependence**

Dependency analysis means checking:Which operations depend on which other operations?

Sometimes two tasks don't depend on data, but they need the same hardware/resource.

Conditions of Parallelism: Data & Name Dependence

Assume task S2 follows task S1 in sequential program order

True Data or Flow Dependence: Task S2 is data dependent on task S1 if an execution path exists from S1 to S2 and if at least one output variable of S1 feeds in as an input operand used by S2

Represented by S1 ⟶ S2 in task dependency graphs

Anti-dependence: Task S2 is anti-dependent on S1, if S2 follows S1 in program order and if the output of S2 overlaps the input of S1

Represented by S1 ⟝⟶ S2 in dependency graphs

s1 want to read before s2 write

Output dependence: Two tasks S1, S2 are output dependent if they produce the same output variables (or output overlaps)

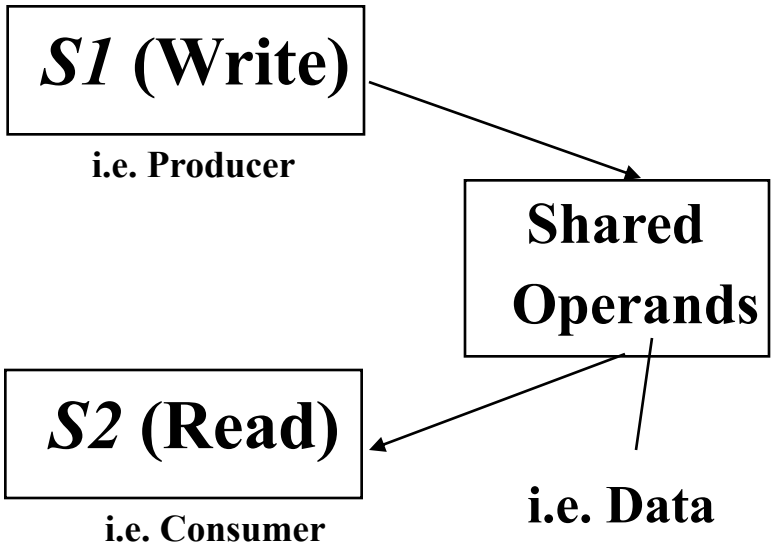
Represented by S1 ⟲⟶ S2 in task dependency graphs

(True) Data (or Flow) Dependence

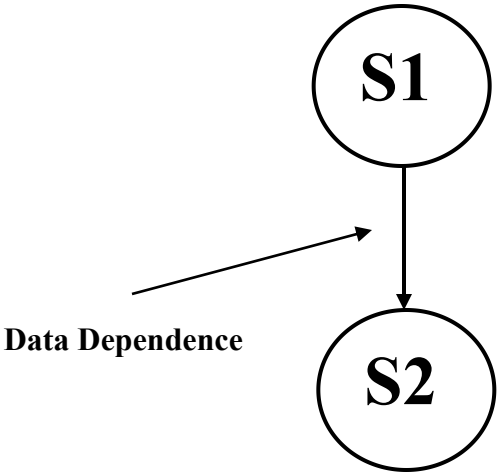
needs a piece of data that a previous instruction is supposed to create

- **Assume task S2 follows task S1 in sequential program order**
- **Task S1 produces one or more results used by task S2,**
  - Then task S2 is said to be data dependent on task S1

S1 ⟶ S2



Task Dependency Graph Representation

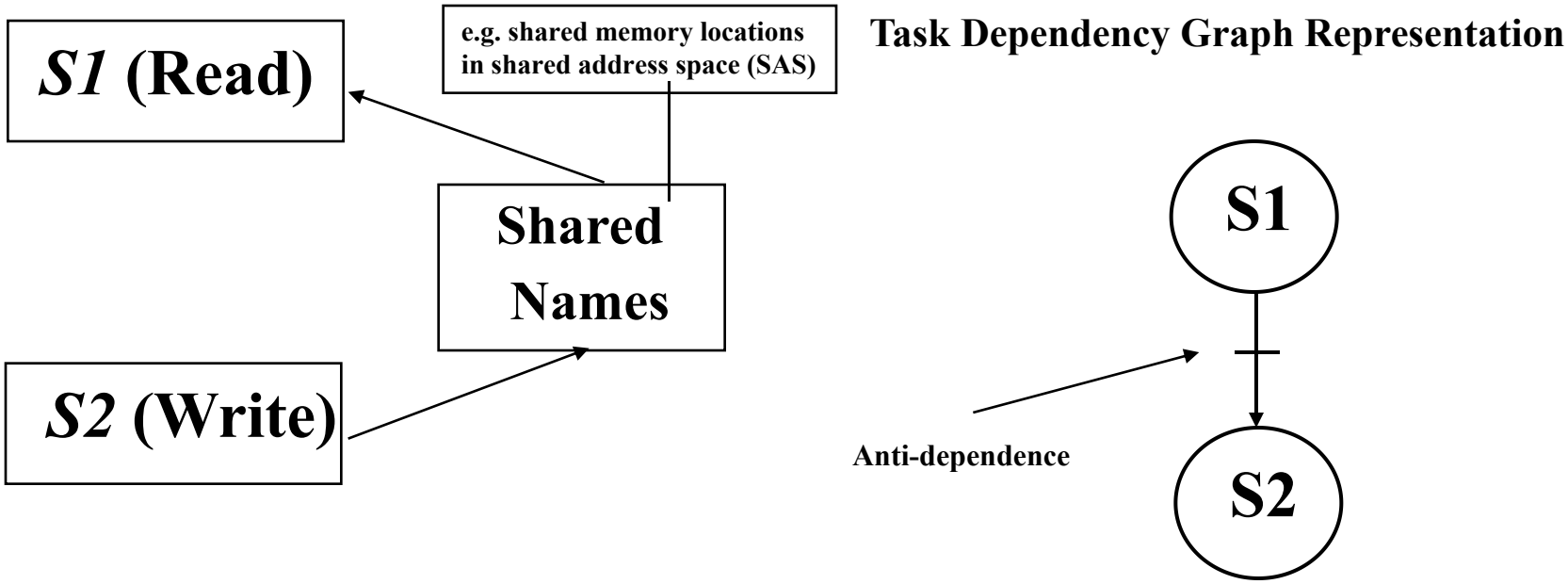


Name Dependence Classification: Anti-Dependence

- Assume task S2 follows task S1 in sequential program order
- Task S1 reads one or more values from one or more names (registers or memory locations)
- Task S2 writes one or more values to the same names (same registers or memory locations read by S1)
  - Then task S2 is said to be anti-dependent on task S1

$S1 \dashrightarrow S2$

Don't overwrite that data yet! I'm still reading it.



Task S2 is anti-dependent on task S1

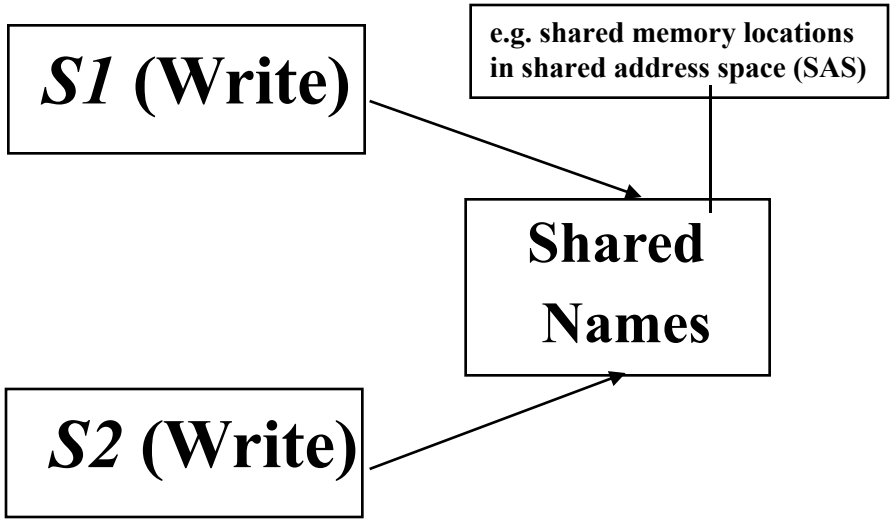
Name Dependence Classification:

Output (or Write) Dependence

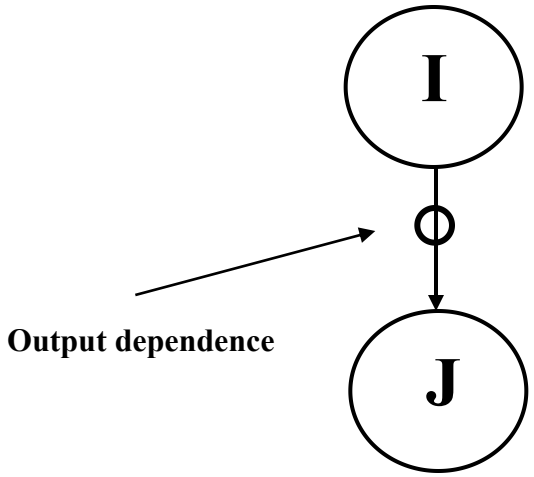
- **Assume task S2 follows task S1 in sequential program order**
- **Both tasks S1, S2 write to the same a name or names (same registers or memory locations)**
  - Then task S2 is said to be output-dependent on task S1

S1 δ^{o} S2

Output dependency is when two commands want to save their answers to the same place



Task Dependency Graph Representation



Task S2 is output-dependent on task S1

Dependency Graph Example

Here assume each instruction is treated as a task:

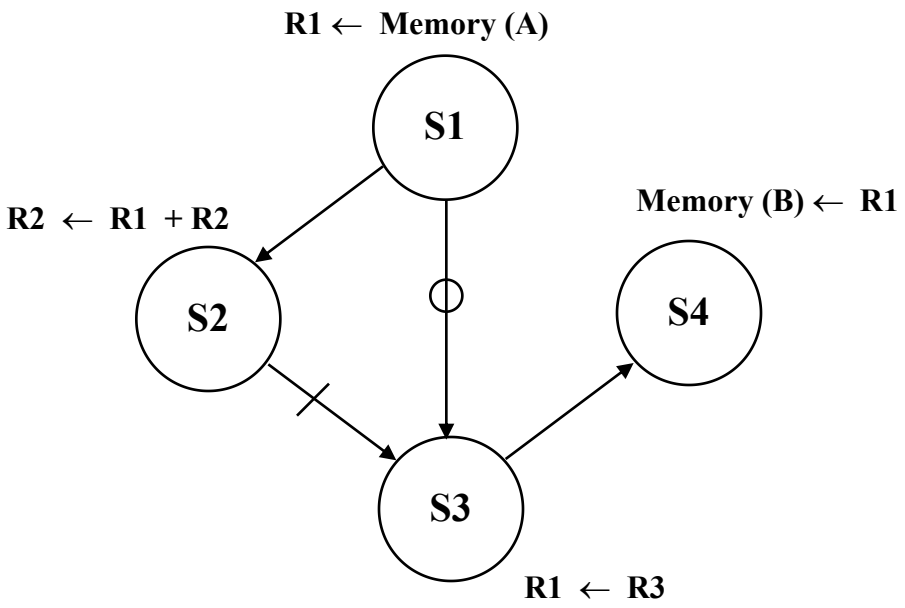
S1:	**Load R1, A**	**/ R1 ← Memory (A) /**
S2:	**Add R2, R1**	**/ R2 ← R1 + R2 /**
S3:	**Move R1, R3**	**/ R1 ← R3 /**
S4:	**Store B, R1**	**/Memory (B) ← R1 /**

Sequential Order

True Date Dependence:
(S1, S2) (S3, S4)
i.e. S1 ⟶ S2
S3 ⟶ S4

Output Dependence:
(S1, S3)
i.e. S1 ⟶ S3

Anti-dependence:
(S2, S3)
i.e. S2 ⟶ S3

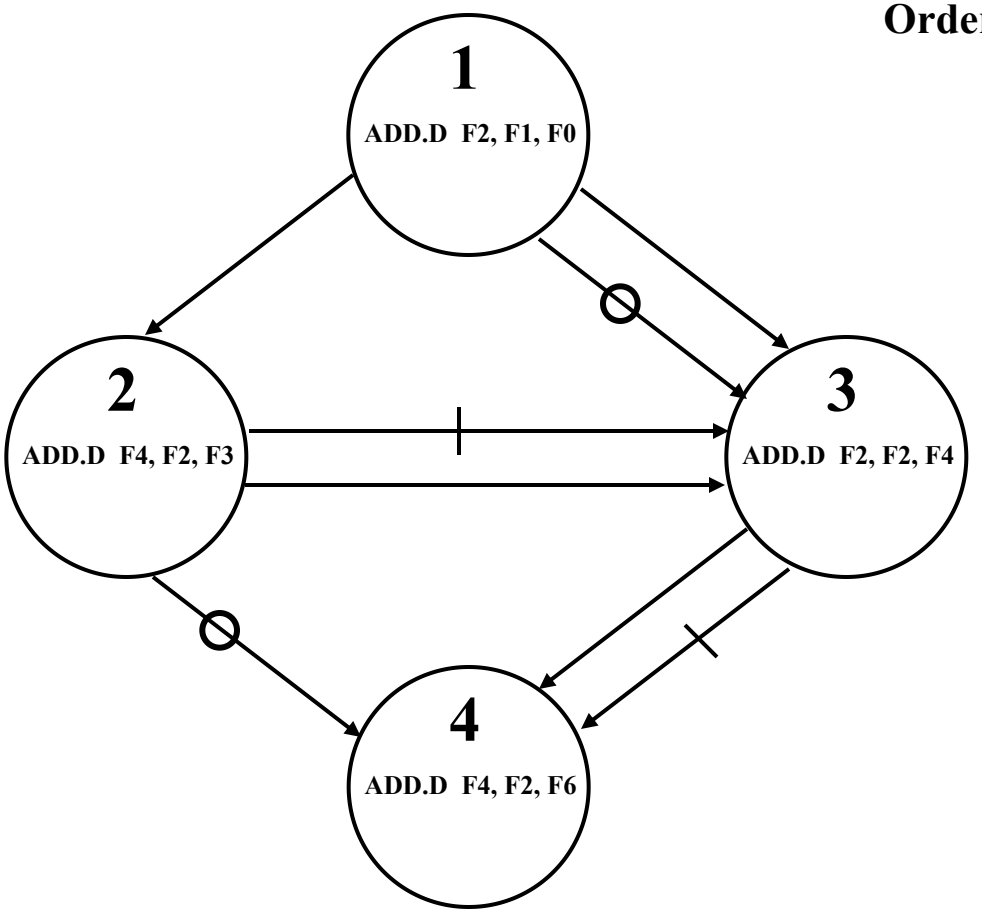


Dependency graph

Dependency Graph Example

Here assume each instruction is treated as a task

Task Dependency graph



Order

MIPS Code

```
1    ADD.D   F2, F1, F0
2    ADD.D   F4, F2, F3
3    ADD.D   F2, F2, F4
4    ADD.D   F4, F2, F6
```

True Date Dependence:

(1, 2) (1, 3) (2, 3) (3, 4)

i.e. 1 → 2 1 → 3
2 → 3 3 → 4

Output Dependence:

(1, 3) (2, 4)

i.e. 1 ⊖→ 3 2 ⊖→ 4

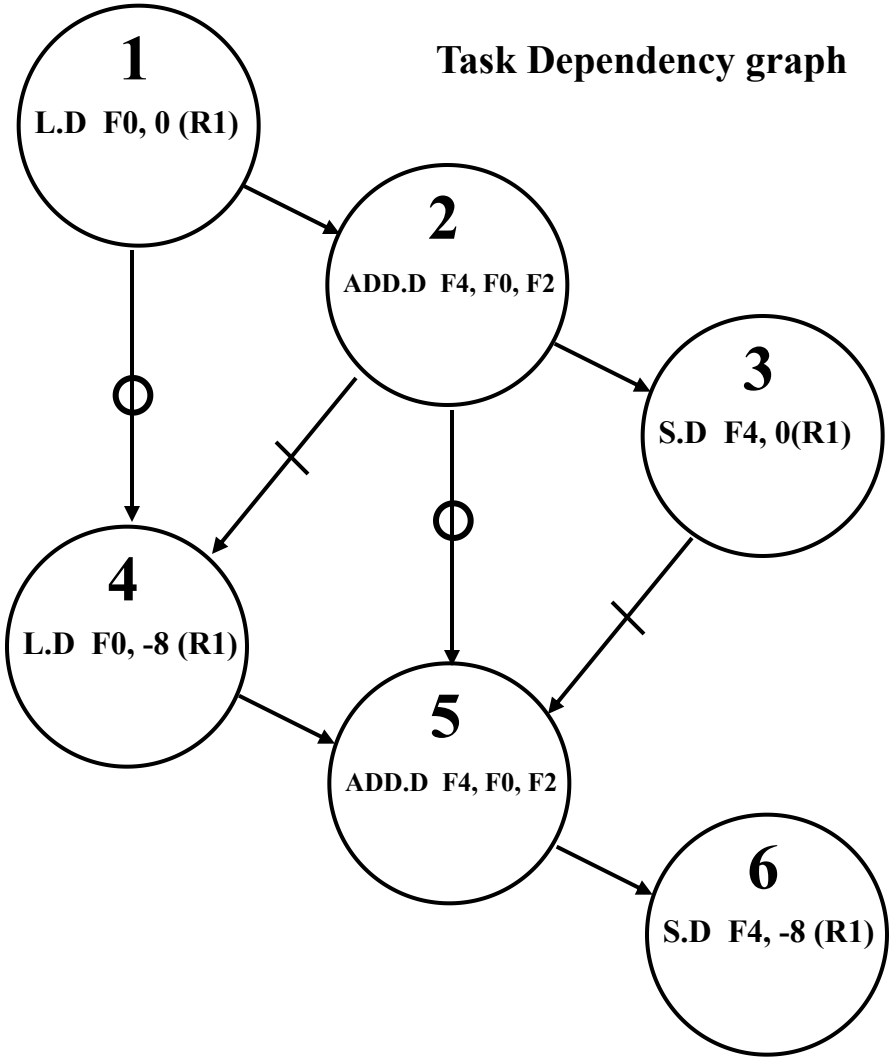
Anti-dependence:

(2, 3) (3, 4)

i.e. 2 ⊣→ 3 3 ⊣→ 4

Dependency Graph Example

Here assume each instruction is treated as a task



MIPS Code

1	L.D	F0, 0 (R1)
2	ADD.D	F4, F0, F2
3	S.D	F4, 0(R1)
4	L.D	F0, -8(R1)
5	ADD.D	F4, F0, F2
6	S.D	F4, -8(R1)

True Date Dependence:
(1, 2) (2, 3) (4, 5) (5, 6)
i.e. 1 → 2 1 → 3
4 → 5 5 → 6

Output Dependence:
(1, 4) (2, 5)
i.e. 1 ⊖→ 4 2 ⊖→ 5

Anti-dependence:
(2, 4) (3, 5)
i.e. 2 ⊣→ 4 3 ⊣→ 5

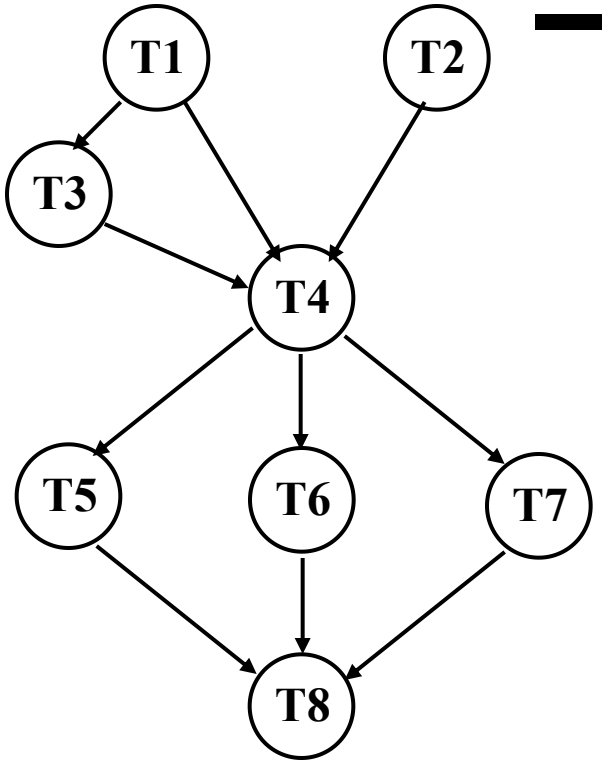
Example Parallel Constructs: Co-begin, Co-end

all execute parallely

- A number of generic parallel constructs can be used to specify or represent parallelism in parallel computations or code including (Co-begin, Co-end).
- Statements or tasks can be run in parallel if they are declared in same block of (Co-begin, Co-end) pair. Start these operations together and wait until all of them finish.
- Example: Given the the following task dependency graph of a computation with eight tasks T1-T8:

Possible Sequential Code Order:

T1
T2
T3
T4
T5,
T6,
T7
T8



Task Dependency Graph

Parallel code using Co-begin, Co-end:

Co-Begin
    T1, T2
Co-End
T3
T4
Co-Begin
    T5, T6, T7
Co-End
T8

Conditions of Parallelism

- **Control Dependence:**
  - **Order of execution cannot be determined before runtime due to conditional statements.**
- **Resource Dependence:**
  - **Concerned with conflicts in using shared resources among parallel tasks, including:**
    - **Functional units (integer, floating point), memory areas, communication links etc.**
- **Bernstein's Conditions of Parallelism:**

Two processes P_1 , P_2 with input sets I_1, I_2 and output sets O_1, O_2 can execute in parallel (denoted by $P_1 \parallel P_2$) if:

$$I_1 \cap O_2 = \varnothing$$

$$I_2 \cap O_1 = \varnothing$$

$$O_1 \cap O_2 = \varnothing$$

i.e no output dependence

If two operations don't interfere with each other's data, they can potentially run in parallel.

Bernstein's Conditions: An Example

- For the following statements P_1, P_2, P_3, P_4, P_5 :
  - Each instruction requires one step to execute
  - Two adders are available

```
P1
Co-Begin
    P1, P3, P5
Co-End
P4
```

$P_1 : C = D \times E$

$P_2 : M = G + C$

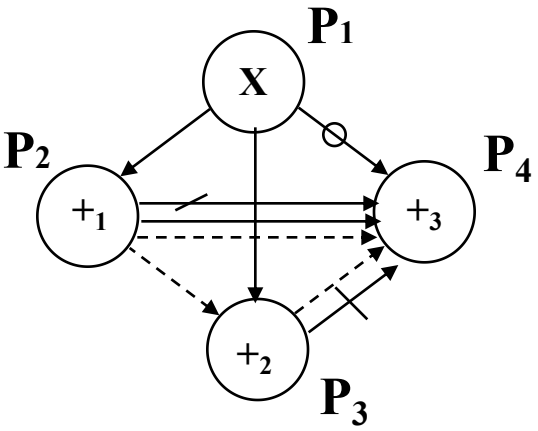
$P_3 : A = B + C$

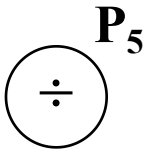
$P_4 : C = L + M$

$P_5 : F = G \div E$

Using Bernstein's Conditions after checking statement pairs:

$P_1 \parallel P_5$, $P_2 \parallel P_3$, $P_2 \parallel P_5$, $P_3 \parallel P_5$, $P_4 \parallel P_5$



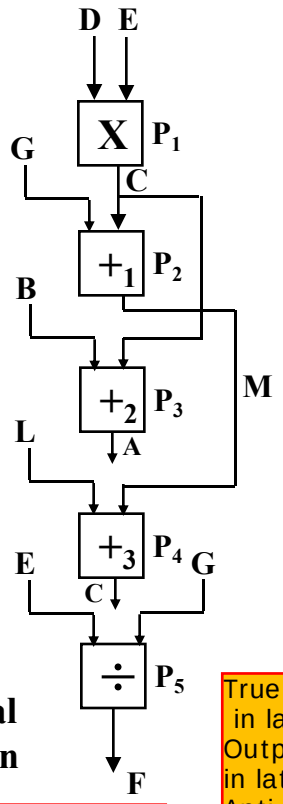


Dependence graph:

Data dependence (solid lines)

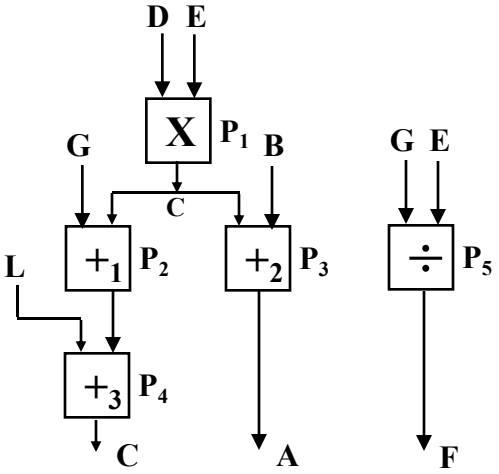
Resource dependence (dashed lines)

look for instructions that use the same mathematical operation or hardware (like two additions), and draw a dashed arrow from the first one to the second one to show they must wait their turn to use it.



Sequential execution

Time

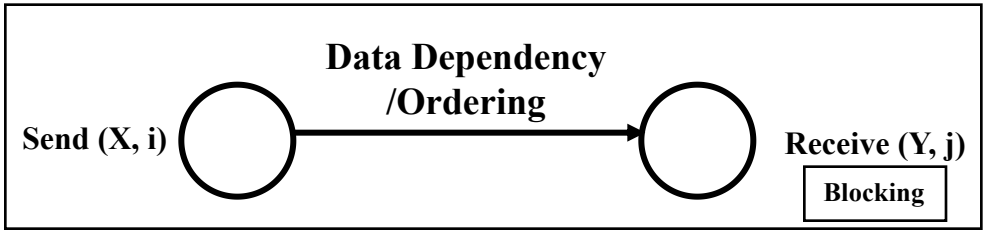


Parallel execution in three steps assuming two adders are available per step

True Dependency: If previous generated data is used in later.
Output Dependency: If same data is generated new in later which is already previously created.
Anti Dependency: If a value is used in previous but is again generated newly.

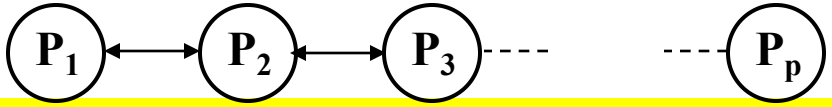
Network Model of Message-Passing Multicomputers

- A network of processors can viewed as a graph G (N, E)
  - Each node $i \in N$ represents a processor
  - Each edge $(i, j) \in E$ represents a two-way communication link between processors i and j.
  - Each processor is assumed to have its own local memory.
  - No shared memory is available.
  - Operation is synchronous or asynchronous (using message passing).
  - Basic message-passing communication primitives:
    - send(X, i) a copy of data X is sent to processor P_i, execution continues.
    - receive(Y, j) execution of recipient processor is suspended (blocked or waits) until the data from processor P_j is received and stored in Y then execution resumes.



Network Model of Multicomputers

- **Routing is concerned with delivering each message from source to destination over the network.**
- **Additional important network topology parameters:**
  - **The network diameter is the maximum distance between any pair of nodes (in links or hops).**
  - **The maximum degree of any node in G**
    - **Directly connected to how many other nodes**
- **Example:**



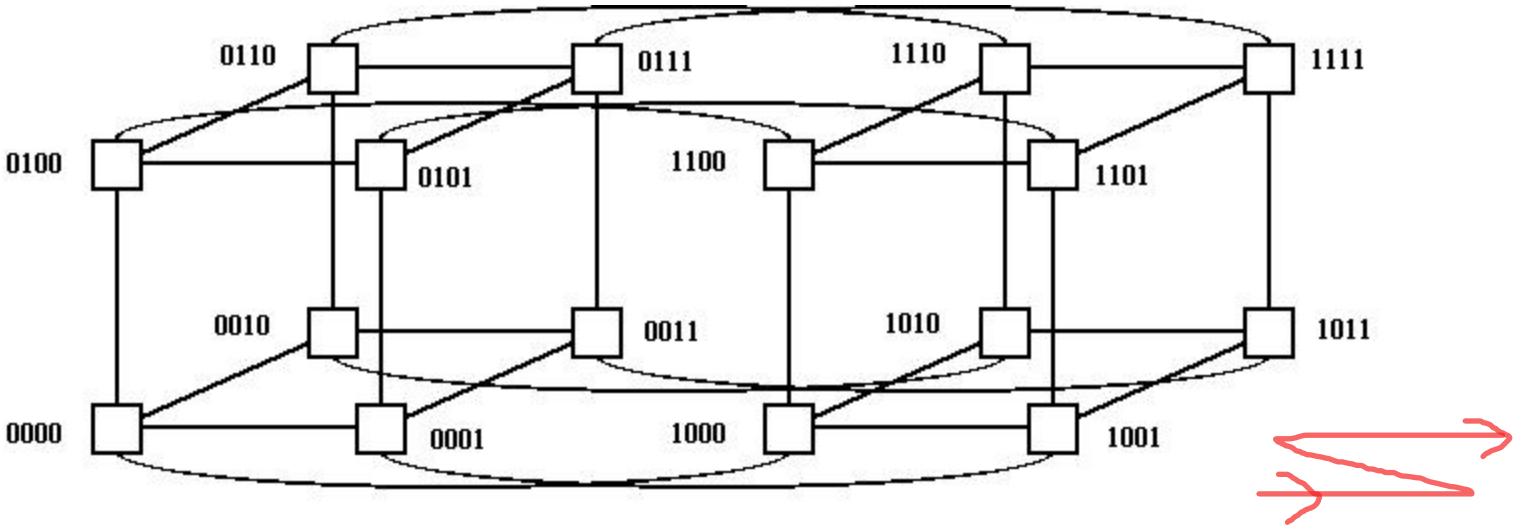
  - **Linear array: P processors P_1, …, P_p are connected in a linear array where:**
    - **Processor P_i is connected to P_{i-1} and P_{i+1} if they exist.**
    - **Diameter is p-1; maximum degree is 2 (1 or 2).**
  - **A ring is a linear array of processors where processors P_1 and P_p are directly connected. Degree = 2, Diameter = p/2**

Example Network Topology:

A Four-Dimensional (4D) Hypercube

Processing elements are connected in a structured way so that communication can happen efficiently.

- **In a d-dimensional binary hypercube, each of the N = 2^d nodes is assigned a d-bit address.**
- **Two processors are connected if their binary addresses differ in one bit position.**
- **Degree = Diameter = d = $\log_2$ N** *here d = 4*



Here: d = Degree = Diameter = 4

Number of nodes = N = $2^d = 2^4$ = 16 nodes

Dimension order routing example node 0000 to 1111:

0000 $\xrightarrow{1}$ 0001 $\xrightarrow{2}$ 0011 $\xrightarrow{3}$ 0111 $\xrightarrow{4}$ 1111 (4 hops)

Creating a Parallel Program

- **Assumption: Sequential algorithm to solve problem is given**
  - **Or a different algorithm with more inherent parallelism is devised.**
  - **Most programming problems have several parallel solutions or algorithms. The best solution may differ from that suggested by existing sequential algorithms.**

One must: Computational Problem ⟶ Parallel Algorithm ⟶ Parallel Program

- **Identify work that can be done in parallel (dependency analysis)**
- **Partition work and perhaps data among processes (Tasks)**
- **Manage data access, communication and synchronization**

Main goal: Maximize Speedup

$$Speedup\ (p) = \frac{Performance(p)}{Performance(1)}$$

By:
1- Minimizing parallelization overheads
2- Balancing workload on processors

For a fixed size problem:

$$Speedup\ (p) = \frac{Time(1)}{Time(p)}$$

The processor with max. execution time determines parallel execution time

$Time\ (p) = Max$ **(Work + Synch Wait Time + Comm Cost + Extra Work)**

Hardware Vs. Software Parallelism

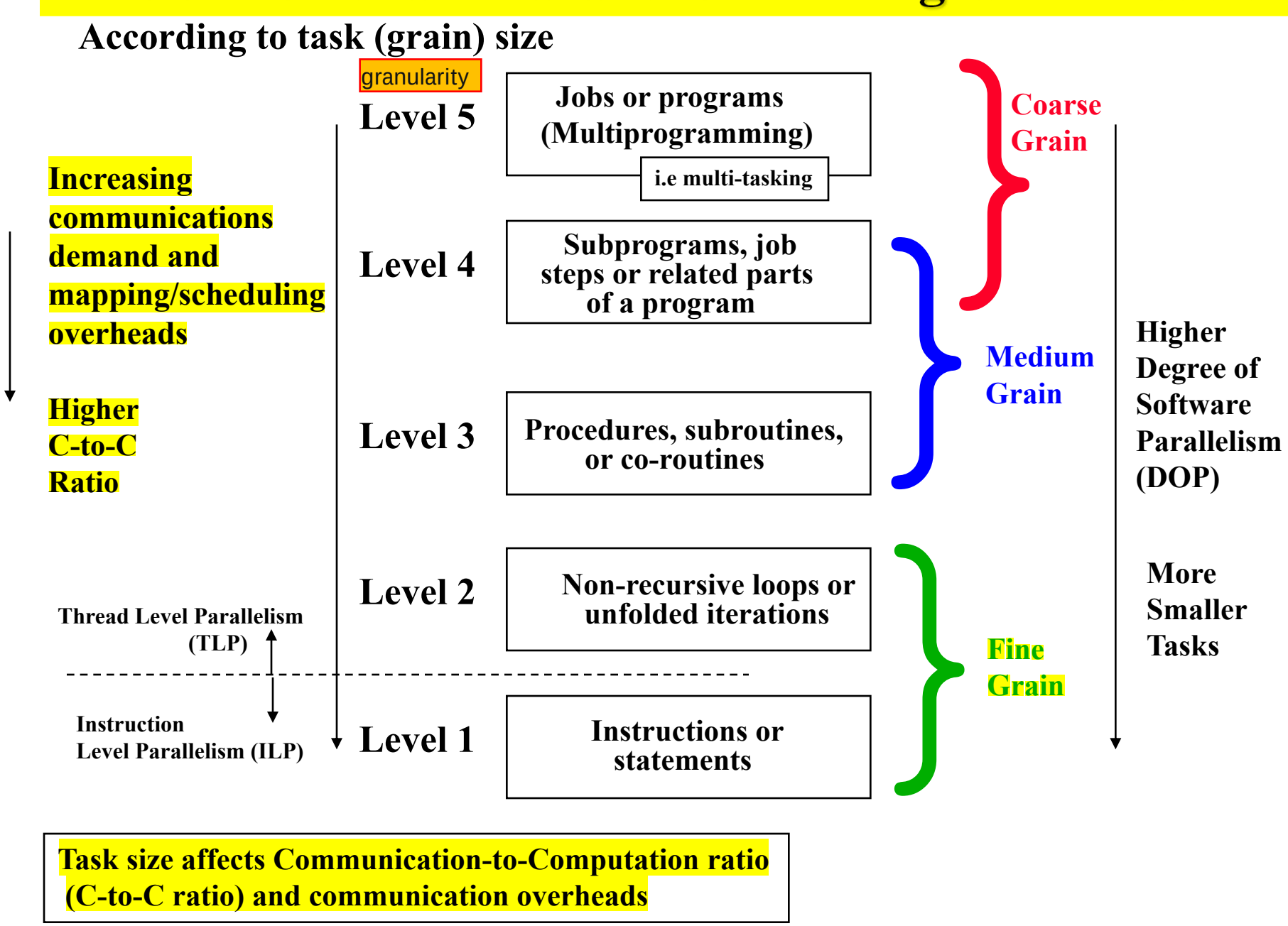
- **Hardware parallelism:** e.g Number of processors
  - Defined by machine architecture, hardware multiplicity (number of processors available) and connectivity.
  - Often a function of cost/performance tradeoffs.
  - Characterized in a single processor by the number of instructions *k* issued in a single cycle (*k*-issue processor).
  - A multiprocessor system with *n* *k*-issue processor can handle a maximum limit of *nk* parallel instructions (at ILP level) or n parallel threads at thread-level parallelism (TLP) level.

- **Software parallelism:** e.g Degree of Software Parallelism (DOP) or number of parallel tasks at selected task at a given time in the parallel computation
  - Defined by the control and data dependence of programs.
  - Revealed in program profiling or program dependency (data flow) graph. → i.e. number of parallel tasks at a given time
  - A function of algorithm, parallel task grain size, programming style and compiler optimization. Grain size means:How much work is inside one parallel task?

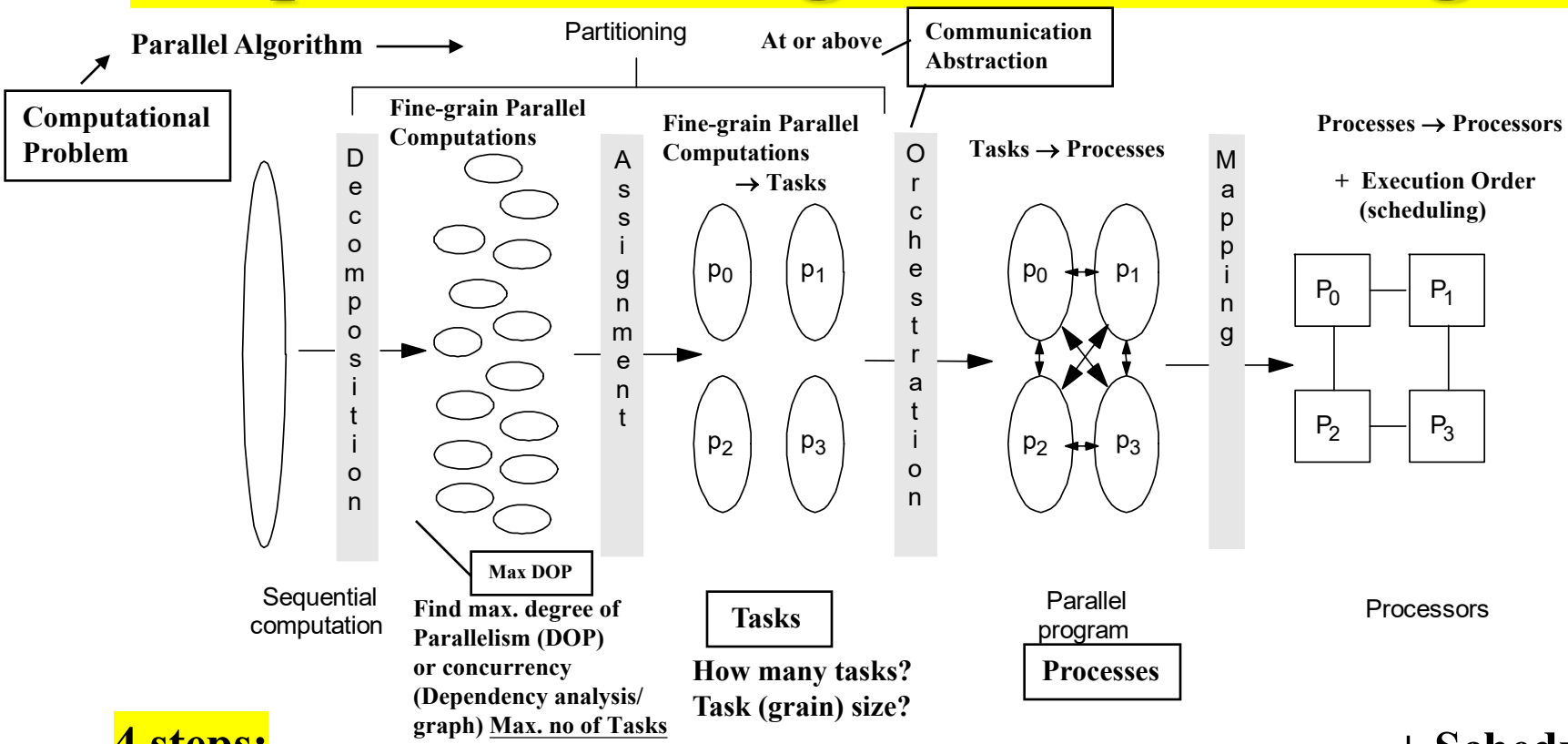
DOP = Degree of Parallelism

How many operations/tasks can be performed simultaneously?

Levels of Software Parallelism in Program Execution



Steps in Creating a Parallel Program



4 steps: **+ Scheduling**

1- Decomposition, 2- Assignment, 3- Orchestration, 4- Mapping

- **Done by programmer or system software (compiler, runtime, ...)**
- **Issues are the same, so assume programmer does it all explicitly**

Computational Problem → Parallel Algorithm → Parallel Program

Partitioning: Decomposition & Assignment

Decomposition — *Dependency Analysis/graph*

- **Break up computation into maximum number of small concurrent computations that can be combined into fewer/larger tasks in assignment step:** (i.e. parallel)
  - Tasks may become available dynamically.
  - No. of available tasks may vary with time.
  - Together with assignment, also called *partitioning*.

(Grain (task) size Problem (Assignment))

i.e. identify concurrency (dependency analysis) and decide level at which to exploit it.

(Task) Assignment

- **Grain-size problem:**
  - To determine the number and size of grains or tasks in a parallel program.
  - Problem and machine-dependent.
  - Solutions involve tradeoffs between parallelism, communication and scheduling/synchronization overheads.
- **Grain packing:** taking a bunch of tiny, fine-grain tasks and "packing" them together into one bigger, coarse-grain task.
  - To combine multiple fine-grain nodes (parallel computations) into a coarse grain node (task) to reduce communication delays and overall scheduling overhead. (i.e larger)

Goal: **Enough tasks to keep processors busy, but not too many (too much overheads)**

- No. of tasks available at a time is upper bound on achievable speedup